

Graphics Unity Project 2026
Split-Screen Game

Example Images



It takes two (Hazelight Studios)

Contents

1	Introduction	4
2	Game Overview & Core Components	5
2.1	Case Studies: Famous Split-Screen Games	5
2.1.1	It Takes Two (2021)	5
2.1.2	Mario Kart 8 Deluxe (2017)	5
2.1.3	Portal 2 (2011)	5
2.1.4	Halo: Combat Evolved (2001)	6
2.2	Playable and Non-Playable Characters	6
2.3	Interactable and Non-Interactable Objects	6
2.4	Commands	7
2.5	Level Design	7
2.6	Camera Control	7
2.7	AI Components	7
2.7.1	AI Navigation	7
2.7.2	AI Behaviors	8
2.8	Audio: Music and Sound Effects	8
2.9	User Interface	9
3	Game Design Document	10
3.1	Example game design document	10
4	Game Logic Flowchart	11
5	Requirements	12
5.1	Basic Requirements (50%)	12
5.2	Extra Requirements (50%)	13
5.3	Any Extra Functionality/Effort (Bonus 10%)	13
6	Basic Implementation Instructions	14
6.1	3D Models and Environment	14
6.2	Physics System	14
6.2.1	Colliders	14
6.2.2	Rigidbody	14
6.3	Scripting and Game Logic	14
6.3.1	Key Lifecycle Methods	14
6.3.2	Component Communication	15
6.4	User Interface (UI)	15
6.5	Audio	15
7	Useful tips	16
8	Project Guidelines	17

1 Introduction

Welcome to the 2026 Graphics Unity Project! This year, we are bringing the excitement of local multiplayer gaming. Your task is to design and develop a **Split-Screen Desktop Game** using the Unity game engine.

In an era dominated by online multiplayer, local split-screen games offer a unique, shared experience, whether it's the high-fives of a cooperative victory or the friendly rivalry of a competitive match. You have the creative freedom to choose the direction of your game:

- **Cooperative (Co-op):** Two players work together to solve puzzles, defeat enemies, or survive against the odds. (e.g., *It Takes Two*, *Portal 2*).
- **Competitive (PvP):** Two players battle against each other for dominance, speed, or high scores. (e.g., *Mario Kart*, *Rocket League*, *GoldenEye 007*).

While split-screen games can be complex, this project focuses on the fundamental mechanics and components. You are not expected to build a AAA title, but rather to demonstrate your understanding of Unity's core systems. You are free to design your own world, characters, and rules. Use your imagination!

A fully completed game should consist of the following sections and Unity components:

1. **Graphics:** 3D scene design, 3D objects, textures, materials, lights.
2. **Physics:** colliders, rigidbodies, forces.
3. **Scripting:** game logic and mechanics, event handling.
4. **Audio:** sound effects, music.
5. **UI:** text display, buttons using Unity's UI components.

2 Game Overview & Core Components

The concept of the Graphics Course's project for 2026 is associated with **Split-Screen Multiplayer Games**. This genre focuses on games where two players share a single screen, typically divided into independent viewpoints, to enable simultaneous local gameplay.

What is a Split-Screen Multiplayer Desktop game?

A split-screen multiplayer desktop game is a type of game where two players engage in gameplay on the same computer, with the screen split to provide each player with their own separate perspective. These games can be competitive, characters play against each other aiming for victory based on score or objective completion, or collaborative, players work together towards a common goal, requiring coordination and shared objectives.

The game typically involves:

- **Independent Player Control:** Each player has full control over their character or entity.
- **Simultaneous Player Input:** Both players provide input concurrently.
- **Shared Environment:** While perspectives are separate, both players exist within the same game world.

2.1 Case Studies: Famous Split-Screen Games

Analyzing successful split-screen games can provide valuable insight into mechanics and level design, whether for cooperative or competitive gameplay.

2.1.1 It Takes Two (2021)

- **Genre:** Action-Adventure Platformer.
- **Description:** A genre-bending adventure where a couple is turned into dolls.
- **Mechanics:** Every level introduces new mechanics that require specific cooperation (e.g., one player has a hammer head, the other has nails).
- It is the gold standard for modern split-screen, forcing players to communicate and work together constantly.

2.1.2 Mario Kart 8 Deluxe (2017)

- **Genre:** Kart Racing.
- **Description:** Players race against each other on various tracks using items to hinder opponents.
- **Mechanics:** Split-screen allows players to see their own view while being aware of their opponent's position.
- A classic example of competitive split-screen where screen peeking is part of the strategy.

2.1.3 Portal 2 (2011)

- **Genre:** Puzzle-Platformer.
- **Description:** Two robots navigate complex test chambers.

- **Mechanics:** Players must use portals to solve physics-based puzzles, often requiring precise timing and trust.
- Perfect example of how to use split-screen for puzzle solving.

2.1.4 Halo: Combat Evolved (2001)

- **Genre:** First-Person Shooter (FPS).
- **Description:** Classic shooter that defined console multiplayer.
- **Mechanics:** Halo's campaign allowed two players to fight through the story together, while multiplayer modes allowed for 4-player split-screen deathmatches.
- The game shows the foundation of split-screen: shared screen real estate and the fun of local social play.

2.2 Playable and Non-Playable Characters

The game will contain the following characters:

- **Playable Characters (PCs).** The PCs are fictional characters, whose actions are controlled by the players. Since this is a split-screen game, there are two distinct PCs (Player 1 and Player 2). They need to cooperate or compete to pass through several levels to fulfill the goal of the game. The PCs enter a scene and need to complete specific objectives (e.g., reach an exit, defeat the opponent, score points) to progress or win. In their way there might be physical obstacles, while the PCs may interact with objects and non-playable characters (NPCs). Each PC has a healthbar and a basic inventory. They can get consumables from the scene, which can be added to their inventory.
- **Non-Playable Characters (NPCs).** Those are game objects with some basic form of artificial intelligence (AI). NPCs can be of different types and have various interactions. e.g., they can interact with the PCs, by helping or attacking them. However, all NPCs must have boundaries based on the type of the aspects of your game.

2.3 Interactable and Non-Interactable Objects

The game will contain the following type of objects:

- **Interactable objects.** Like any other game, there must be some objects that the player can interact with. Those can be movable rocks, unlockable doors, opening chests, growing plants, etc. The task of the player is to interact with those objects.
- **Physical obstacles.** Those are immovable objects that block the path of the player (e.g., trees, rocks, etc.).
- **Consumables & Power-ups.** These are objects located in the scene that apply a temporary or permanent effect to the player. They can be:
 - **Instant:** Activated immediately upon contact (e.g., a speed pad on the ground, a health pack, or a trap that slows movement).
 - **Inventory Items:** Collected and stored for later use (e.g., a potion, a shield, or a projectile).

Consumables can have **positive effects** like increased speed, strength, or invulnerability, or **negative effects** like inverted controls, slowed movement, or vision obstruction.

2.4 Commands

The players must be able to use specific commands. Core commands include, but are not limited to, the following:

- **Move Command:** Each player can move their playable character to a specific destination. This movement is typically done with WASD keys for Player 1 and Arrow Keys for Player 2, or using gamepads.
- **Interact with Object Command:** The playable characters can interact with other objects which are part of a solvable puzzle. Such examples are: grab a rock and move it, use a key on a door to open it etc.
- **Use Consumable Command:** The playable characters can use an item from their inventory or activate a consumable in the environment. This includes consuming a potion, activating a power-up, or triggering a temporary effect. Note that some consumable might be activated automatically upon collision (e.g., walking over a speed pad).
- **Interact with NPC/Player Command:** The playable characters can interact with NPCs or the other player. NPCs can be either hostile or friendly. A classic example is the attack command where a player can attack NPCs or the opponent.

2.5 Level Design

The level design is a scene with a defined area, an entrance or spawn points and exit for the playable character. The scene is filled with physical obstacles, interactable objects, consumables and NPCs. The playable character(s) will enter from the starting position and aim to complete the level's objective. The main objective is to unlock the exit by achieving a set of conditions like solving a puzzle or defeating the opponent.

2.6 Camera Control

In a split-screen game, the camera system is crucial for providing each player with a clear view of their character and surroundings. You must implement a dual-camera setup:

- **Split-Screen Setup:** You need two separate Camera objects in your scene.
 - **Player 1 Camera:** Renders to the left (or top) half of the screen.
 - **Player 2 Camera:** Renders to the right (or bottom) half of the screen.
- **Camera Movement:** The camera should smoothly follow its assigned player. Depending on your game design, you may allow players to rotate the camera around their character (e.g., using designated controls for each player) to get a better view of the environment.

2.7 AI Components

Non-Playable Characters (NPCs) bring life to your game world. They should exhibit intelligent behavior, such as navigating the environment and reacting to the players.

2.7.1 AI Navigation

NPCs must be able to move around the level intelligently without walking through walls or falling off the map. To achieve this, you should use **Unity's NavMesh System**:

- **Bake the NavMesh:** Mark your floors and ramps as "Navigation Static" and bake the NavMesh to define walkable areas.

- **NavMesh Agent:** Add this component to your NPCs so they can calculate paths and move towards a target destination automatically avoiding obstacles.

2.7.2 AI Behaviors

NPCs should follow a logic structure, often called a **Finite State Machine (FSM)**, to decide what to do based on the game context.

Common NPC States:

- **Idle:** The NPC stands still, looks around, or plays an idle animation.
- **Patrol / Wander:** The NPC moves between predefined points or wanders randomly within an area.
- **Approach / Follow:** If a player enters a “detection radius”, the NPC moves towards them. This could be a monster chasing a player or a pet following its owner.
- **Action / Interaction:** When close enough, the NPC performs a specific action. This could be attacking, starting a dialogue, trading items, healing the player or something entirely different (Up to you!).

Example Scenarios:

- **Hostile (Guard):** Patrols a hallway. If it spots a player, it switches to *Chase*. Upon catching the player, it switches to *Attack*.
- **Friendly (Shopkeeper):** Stands in *Idle* behind a counter. When a player approaches, it switches to *Greet/Trade*, opening a shop UI.
- **Quest NPC:** Stands in a specific location. When a player approaches, it switches to *Dialogue*, allowing the player to receive or complete quests.
- **Neutral (Pet):** Wanders around (*Patrol*). When the player calls it, it switches to *Follow*.

2.8 Audio: Music and Sound Effects

Audio is a vital component that breathes life into your game, setting the mood and providing feedback to the player. You should implement two distinct types of audio:

- **Background Music:** A looping track that sets the atmosphere of the level (e.g., tense music for a boss fight, calm music for a puzzle).
- **Sound Effects (SFX):** Short audio clips triggered by specific events to provide immediate feedback. Examples include:
 - **Interaction:** Picking up a coin, opening a door, pressing a button.
 - **Movement:** Footsteps, jumping, landing.
 - **Combat/Action:** Swinging a sword, shooting, taking damage.

Unity Implementation Tip: Use **Audio Source** components to play sounds. For background music, set the Audio Source to “Loop”. For SFX, use `PlayOneShot()` in your scripts. (Optional) Consider using **3D Sound** settings for in-game objects so players can hear where a sound is coming from (spatial audio).

2.9 User Interface

You should implement the basic UI functionality required for your game to be played smoothly. Don't take anything for granted and explain everything in your game using UIs or by guiding your players in a smart way. Show at every stage of the game the progression to the players, and communicate with them every info that affects their experience throughout the entire procedure (e.g., show healthbar, available consumables, etc). Since this is a split-screen game, ensure UI elements are positioned correctly for each player's view.

3 Game Design Document

The core game design dictates what the player expects to see in the game in three sentences. It is a very broad guideline that dictates the core design principles of the development process and helps guide us when coming up with new ideas or when searching for assets.

The core design includes four topics: Playable Characters (Who are the characters controlled by the players?), Setting (In what location and time period does the game take place? What is the main visual theme or technological background?), Goal (what do the players seek to achieve and how?) and Scenario (what happens in the game?).

This is a mandatory component of the project submission that you **must** implement.

3.1 Example game design document

- **Game Name:** Ancient Egypt
- **Playable Character:** The player controls a tomb rider character located in ancient Egypt.
- **Setting:** Early 90s. African desert.
- **Goal:** The player must solve puzzles involving statues in order to find the hidden treasure which is located in the final level. The player has to move all the statues in the correct pavement in order to progress to the next level. When all statues are placed in the correct pavement the exit is unlocked and the playable character can move to the next level.
- **Scenario:** The playable character is constantly chased by mummies (hostile NPC). The playable character holds a torch. The brightness of the torch runs out both over time and if the playable character is attacked by mummies. If the torch is extinguished the player loses the game and has to start over. There is an egyptian priest (friendly NPC) who heals the playable character when he is near the playable character.

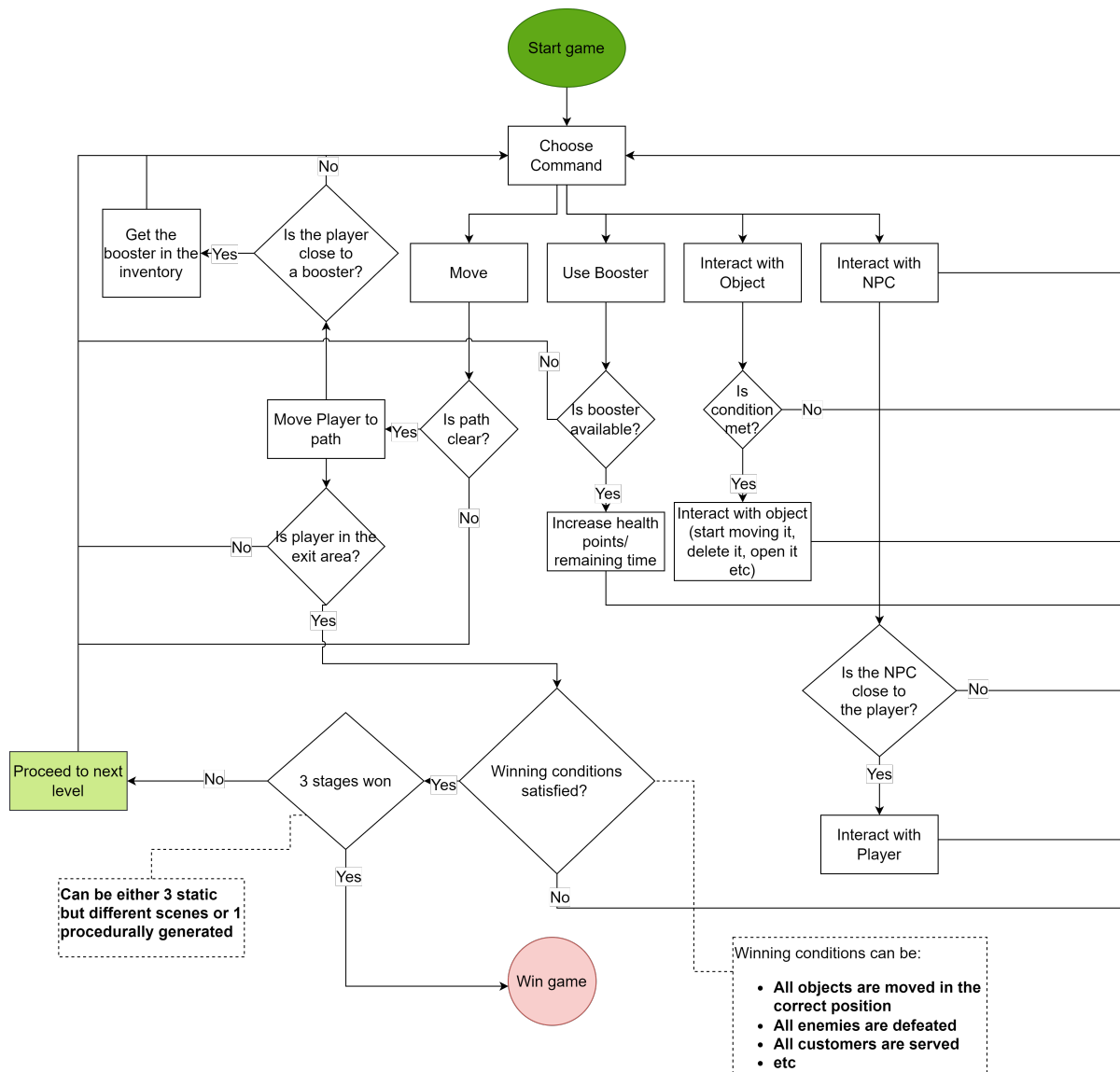
Hostile NPC: Mummy	Monster that tries to attack the playable character.
Friendly NPC: Egyptian Priest	The priest heals the playable character when he is nearby.
Interactable Object: Statue	A statue that can be moved by the playable character. Must be placed in the correct pavement.
Obstacle: Palm Tree	A tree that blocks the movement of the playable character.
Obstacle: Rock	A rock that also blocks the movement of the playable character.
Obstacle: Entrance	The entering point of the playable character to the scene.
Obstacle: Exit	The exiting point of the playable character from the scene.
Booster: Torch	The torch extends the remaining time of the playable character.

4 Game Logic Flowchart

A game logic flowchart is a visual diagram that outlines how a game operates, showing the sequence of events, player actions, decisions, and outcomes. It maps out the flow from starting the game, through gameplay loops (such as player input, game state updates, and win/lose conditions), to the end of the game. This helps developers clearly design and organize how the game behaves and responds at each step. Below is a flowchart that can work as a template for your game logic design.

This is a mandatory component of the project submission that you **must** implement.

You may open a unique copy of this template in your browser and edit it (functionality provided by draw.io) by following this link: [here](https://draw.io)



Game Logic Template

5 Requirements

You must complete the implementation of minimum requirements first, in order to start the implementation of the extra requirements.

5.1 Basic Requirements (50%)

In order to pass the Graphics course (5/10) you have to implement at least the minimum requirements. Additionally, to receive points from the extra requirements, all basic requirements must be implemented.

Game design document & Game logic flowchart: In order to pass the project of Graphics 2026 it is mandatory for your report to contain a game design document and a flowchart of the game logic. **You cannot pass the course without it.**

1. **Graphics (10%):** One (or more) Unity 3D scene(s) with the necessary 3D objects. All objects must have different materials with at least one basic texture on them. You can use Unity's 3D models (cubes, spheres, ..) to create your scene by transforming them to give them the shape you wish. You could also search third-party 3d models (from Sketch-fab or Unity's Asset Store) and import them in your project.
2. **Physics (5%):** All objects must have colliders and rigidbodies so that they behave as naturally and realistically as possible.
3. **Level Design (10%):** Three different playable levels. The difference between scenes should at least be the location of the NPCs, the consumables and the obstacles.
4. **Game Logic and Mechanics (10%):** At the minimum requirements, your project must be able to provide the following functionality:
 - Two playable characters that move in the plane.
 - At least two types of consumables that increase some useful stat of the player or decrease a stat of the enemy.
 - At least one NPC that can interact with or attack the player.
 - A game manager who will be monitoring the game logic state and check whether finishing conditions are met.
5. **UI & Cameras (10%):** The UI components need to be handled by scripts so they are updated in real-time.
 - When the game first opens, it should start with a main menu with at least one button to start the game, one button containing a quick textual tutorial of the game and another to quit the game. The main menu should also have a credits button, containing a textual acknowledgment section. Credits should contain all sources of 3D Objects, Music and other components, used in your project.
 - At least two UI text elements presenting useful information for each playable character (e.g., the healthbar and the available consumables in the inventory).
 - For each player, at least one **Camera** must exist in the scene.
6. **Audio (5%):** At least one **Audio source** playing an ambient sound.

5.2 Extra Requirements (50%)

You must complete the minimum requirements first, before starting the extra requirements.

1. **Graphics (10%):** The PCs and NPCs must have some basic animations for walking and interacting with the players. e.g., attacking or trading with the players, being happy with the food, executing a transaction etc.
2. **Level Design (10%):** One Procedural/Fully Random Generated Map. Procedural means objects are generated by an algorithm (procedure) that uses randomness. Fully random generated maps means objects are placed independently at random. To receive full marks for this component, the placement of the NPCs, the consumables and the obstacles in the scene should at least be procedurally generated.

NOTE: If a Procedural Generated Map is successfully implemented, it will automatically fulfill the **Basic Requirement #3 (Level Design)**, and you will receive the full 10% for that section without the need to provide three separate static levels.

3. **Game Logic and Mechanics (20%):**
 - A second distinct playable character for each player, with at least one different command or ability from the first (e.g., one can jump and the other one can move objects) and with a different 3D model. This can be done either by in-game character switching or with a separate character selection panel in the main menu.
 - More than one type of NPC with different mechanics.
4. **UI & Cameras (5%):** More than one camera point of view. Each player must have the ability to change their camera point of view independently from the other player.
5. **Audio (5%):** Sound effects when something happens (e.g., opening a door, cutting carrots, hitting an enemy).

5.3 Any Extra Functionality/Effort (Bonus 10%)

- Bonus can also be given in cases where the implementation of a feature is extremely well-developed or very creative/unique.
- Self made 3D/2D assets, Music, Videos count as extra effort and give extra bonus. You must clearly state which assets were custom made and the software you made them with in your credits.

6 Basic Implementation Instructions

This section provides a guide to the fundamental Unity concepts you will need to implement your project. For detailed information, always refer to the [Unity Manual](#) and [Scripting API](#).

6.1 3D Models and Environment

Creating a visually appealing world is the first step in game development. You have two main options for acquiring 3D assets: creating them yourself or importing them.

If you are comfortable with 3D modeling software like Blender, you can create custom models and import them into Unity. For those who prefer to stay within the Unity Editor, the [ProBuilder](#) package is an excellent tool for gray-boxing and creating level geometry directly in the scene. Alternatively, you can download high-quality assets from the [Unity Asset Store](#) or websites like [Sketchfab](#). Remember to credit any third-party assets you use.

To make your objects look realistic, you must apply **Materials**. A Material defines how a surface renders, including its color, texture, and smoothness. You can create a new Material in the Project window (*Create > Material*) and drag it onto your object.

[Unity Documentation: Materials](#)

6.2 Physics System

Unity's built-in physics engine handles collisions and movement simulation. Understanding the relationship between Colliders and Rigidbodies is crucial.

6.2.1 Colliders

A **Collider** component defines the physical shape of an object for collision purposes. It does not need to match the visual mesh exactly; often, a simple primitive (Box, Sphere, Capsule) is more efficient and sufficient for gameplay. Without a Collider, objects will pass through each other like ghosts.

[Unity Documentation: Colliders](#)

6.2.2 Rigidbodies

The **Rigidbody** component enables an object to be affected by physical forces, such as gravity and collisions. If you want an object to fall, be pushed, or interact physically with the player, it must have a Rigidbody. You can adjust parameters like **Mass** (how heavy it is) and **Drag** (air resistance) to tune the feel of the object.

[Unity Documentation: Rigidbody](#)

6.3 Scripting and Game Logic

Scripts are the brain of your game. Written in **C#**, they control object behavior, handle input, and manage game rules. In Unity, scripts typically derive from the **MonoBehaviour** class, which allows them to attach to GameObjects and execute code during specific game events.

6.3.1 Key Lifecycle Methods

- **Start()**: Called once when the script instance is enabled, before the first frame update. Use this for initialization, such as setting up variables or finding references to other objects.
- **Update()**: Called once per frame. This is where you handle user input (e.g., `Input.GetAxis`) and non-physics logic (e.g., timers, simple movement).

- **FixedUpdate():** Called at fixed time intervals. All physics-related calculations (e.g., adding force to a **Rigidbody**) should happen here to ensure consistent simulation.

6.3.2 Component Communication

Scripts often need to interact with other components on the same object or different objects. The **GetComponent<Type>()** method is the primary way to access other components. For example, to make a character jump, your script would get a reference to the **Rigidbody** component and apply an upward force.

[Unity Documentation: Scripting Overview](#)

6.4 User Interface (UI)

The UI system allows you to display information like health bars, scores, and menus. Unity uses a **Canvas** system to render 2D elements over the 3D game world. Key components include **Text** (for displaying strings), **Images** (for icons and bars), and **Buttons** (for interaction). You can anchor these elements to different parts of the screen to ensure they look correct on different resolutions.

[Unity Documentation: User Interface](#)

6.5 Audio

Sound is essential for immersion. The audio system relies on two main components:

- **Audio Listener:** Acts as the “ears” of the world. By default, this is attached to the Main Camera.
- **Audio Source:** Emits sound. You attach this to objects that make noise (e.g., the player, an explosion, a radio).

You can play sounds automatically on start (for background music) or trigger them via scripts using methods like **Play()** or **PlayOneShot()**.

[Unity Documentation: Audio](#)

7 Useful tips

- It is highly recommended to use the Unity [Documentation](#) as it provides both explanations on all of Unity's tools as well as examples on how to use them. When you open the Documentation remember to select your Unity Version on the top left.
- It is highly recommended to give your executable version of your game to be tested by others without your guidance. Try to evaluate your game utilizing the thinking-aloud method. In a thinking-aloud test, you ask test participants to use the system while continuously thinking out loud — that is, simply verbalizing their thoughts as they move through the user interface.
 - Don't trust your instincts on the quality of your UIs implementation. User experience is crucial during the testing of your game. Put effort into the creation of your UI/UX and make it as intuitive as possible.
- During the exam, you will be graded based on the playable version of your game. Anything you implement that cannot be shown while playing the game will not be graded. As such you should take into consideration:
 - Someone playing the game should be able to see everything you want to show in a maximum of 10 minutes. You will be able to give us advice while playing your game in order to achieve this.
- Excluding extreme circumstances, we will not view your project through the Unity Editor so make sure your build is fully playable! Some common issues that may appear include the following:
 - UI elements may not scale correctly with your screen resolution and be out of the playable field. Make sure your game plays correctly at least for the recommended resolution of Full HD (1920x1080) or smaller. If you have a specific resolution you tested your game at and it works correctly other than Full HD specify it on your report.
 - Sometimes things that work in the Editor do not work the same way after you build the game. Always build and test your game during development! The most common issues involve the initialization of objects while switching between scenes e.g. when transitioning from the main menu to the game or when restarting the game and reloading the same scene.
- It is recommended you watch online tutorials and even exchange ideas with each other but plagiarism is strictly forbidden. Additionally, if you get some piece of code from the internet and you cannot explain it, you will not receive any points for the respective component.

8 Project Guidelines

- Your code must be organized in different scripts, depending on the functionality. **Each script must have a distinct role.**
- Use script names that make sense and attach them to related game objects. For example, create a "UIManager" script to handle the game UI, a "GameManager" script to hold variables and objects that keep track of the game's state.
- Your code must be written in C# programming language ONLY.
- Organize your game objects in a hierarchy that makes sense and is easy to understand. For example, if you have game objects such as "Cube(1)", "Cube(2)", "Cube(3)", ... , make them children of a single "CUBES" game object.
- Organize your asset files (textures, 3d models, materials, scripts, sounds, ...) in separate folders. Obviously, the name of the folder should be representative of the content.
- You may use the Internet (YouTube, blogs, ...) for tutorials and code examples, but copy-pasting entire code or functionality is obvious and will NOT be accepted.
- You cannot use "Complete Game Kits" or plugins that do everything for you (e.g., a "FPS All-in-One Kit" that handles movement, shooting, and AI automatically).

Gamepad Support Please be advised that while you may choose to implement gamepad support for split-screen controls, it is mandatory to also provide keyboard and mouse shortcuts to ensure your game can be tested fairly by the instructors. Additionally, please note that implementing these alternative controls for the camera setup will not grant any extra points for the camera component or bonus marks beyond the basic requirements.

LLM Usage Policy Use of LLMs (e.g., ChatGPT, Claude, Gemini, Copilot) is permitted for generating boilerplate code and assets, debugging, and assisting with technical formulas. However, you are responsible for every line of code in your project. You must be able to explain the logic and integration of any AI-generated components during the evaluation of the project. If you cannot demonstrate a complete understanding of how a piece of code functions, it will be treated as plagiarism and no points will be awarded for that component.

and remember while doing this project to **have fun!**

"A game is a series of interesting choices." — Sid Meier